Functional Programming Practical Week 7

Explicit Outcomes Map and Bind

50 tasks and five worked examples. Complete the class set below; the remaining tasks are additional practice. Project integration tasks can be adapted to your final project.

Class work

Suggested class set: Tasks 01–06, with the stated checks and one additional boundary or failure assertion for each task. The instructor may replace these with tasks suited to the group. Intermediate tasks are further practice. Choose one of Tasks 46–49 for project extension; completing all 50 tasks is not required.

Native tasks require the supplied library and a C/C++ compiler. Complete setup before class or use these as an optional extension. Do not execute broken bindings or unbounded recursion before repairing them.

Use the Checks project as an assertion-runner example. Its Task 01 checks intentionally fail against the broken starter. Repair Task 01, rerun the checks, then add tests for the other selected tasks.

DIFFICULTY	TASKS	COUNT
BASIC	01–20	20
INTERMEDIATE	21–40	20
ADVANCED	41–50	10

TASK FORMAT	COUNT	PURPOSE
FIX CODE	15	Identify the bug and repair observable behaviour
COMPLETE TODO	15	Complete the supplied contract
IMPLEMENT	15	Design a small operation from a stated contract
INTEGRATE	5	Apply the techniques in one project module

Material

Overview

Use Maybe to distinguish absence from valid values such as 0 or false. Use Result for expected failures. Map transforms a successful value; Bind continues with another operation that can fail. Preserve error reasons and skip dependent work after failure. Handle operational exceptions at the I/O boundary.

Maybe and primitive values

Some(0), Some(false), and Some("") are present values. None means absence. If the domain disallows an empty string, validate that separately. A lookup can return None and later become a Result failure when the workflow requires the item.

Result

The teaching model has Ok(T Value) and Fail(string Error). Access Value only after matching Ok. Use Fail for normal rejections such as insufficient stock or an invalid date range. Keep both the success and failure data internally consistent.

Map

Map receives a plain value transformation T -> U and returns Result<U>. For Ok it applies that function; for Fail it preserves the error without calling the function. Mapping a function that returns Result creates a nested wrapper and usually signals that Bind is needed.

Bind

Bind receives T -> Result<U>. It runs the next potentially failing operation after Ok and returns that operation's result. After Fail it preserves the failure. Thus Map changes a value, whereas Bind continues a fallible workflow.

Effects and boundaries

Pure validators and calculations return outcomes. A file boundary can catch an IOException or a parsing exception and convert it to a clear Result. Reporting is a separate effect. Use only relevant catches, and never reinterpret a detected operational problem as an empty successful dataset.

Working conventions

Use the supplied .NET 8 SharedModels.cs. R contains reference Map and Bind implementations used by examples; tasks asking you to implement them require your own methods. Tasks 01 to 20 are basic, 21 to 40 intermediate, and 41 to 50 advanced. Each fix or TODO snippet is a member of its TaskNN class. Tests use assertions, controlled inputs, and temporary files where needed.

Shared teaching types

These are reference and starter types, not a fully validated production domain. Tasks may ask you to strengthen or replace them. Some starter fragments need a different signature after repair. Each TaskNN class is isolated; remove its wrapper when moving operations into your project. Implement-type tasks provide the contract without a solution.

```
public record Product(string Id, int Stock, decimal Price);
public record Quote(string Id, int Quantity, decimal Total);
public abstract record Maybe<T>
{
    private protected Maybe() { }
    public sealed record Some(T Value) : Maybe<T>;
    public sealed record None : Maybe<T>;
}
public abstract record Result<T>
{
    private protected Result() { }
    public sealed record Ok(T Value) : Result<T>;
    public sealed record Fail(string Error) : Result<T>;
}
public static class R
{
    public static Result<U> Map<T,U>(Result<T> r, Func<T,U> f)
        => r switch {
            Result<T>.Ok(var x) => new Result<U>.Ok(f(x)),
            Result<T>.Fail(var e) => new Result<U>.Fail(e),
            _ => throw new ArgumentException("Unknown case") };
    public static Result<U> Bind<T,U>(Result<T> r, Func<T,Result<U>> f)
        => r switch {
            Result<T>.Ok(var x) => f(x),
            Result<T>.Fail(var e) => new Result<U>.Fail(e),
            _ => throw new ArgumentException("Unknown case") };
}
```

Worked examples

Example 01 Presence includes zero

Represent a present primitive separately from an absent value.

```
Maybe<int> present = new Maybe<int>.Some(0);
Maybe<int> absent = new Maybe<int>.None();
Console.WriteLine(Describe(present));
Console.WriteLine(Describe(absent));

public static string Describe(Maybe<int> value) => value switch
{
    Maybe<int>.Some(var x) => $"Present:{x}",
    Maybe<int>.None => "Missing",
    _ => throw new ArgumentException("Unknown case")
};
```

Expected result

```
Present:0
Missing
```

Explanation

1. Some carries a value even when that value is zero. 2. None has no payload. 3. The pattern determines whether a payload can be read. 4. Validation of a domain range is separate from presence.

Example 02 A validator returns a Result

Validate a requested quantity without using an exception for an ordinary invalid request.

```
Console.WriteLine(Describe(Validate(2)));
Console.WriteLine(Describe(Validate(0)));

public static Result<int> Validate(int q) => q > 0
    ? new Result<int>.Ok(q)
    : new Result<int>.Fail("Quantity must be positive");
public static string Describe(Result<int> r) => r switch
{
    Result<int>.Ok(var q) => $"Accepted:{q}",
    Result<int>.Fail(var e) => e,
    _ => throw new ArgumentException("Unknown case")
};
```

Expected result

```
Accepted:2
Quantity must be positive
```

Explanation

1. The input rule is checked before downstream calculation. 2. Ok preserves the accepted value. 3. Fail names the violated rule. 4. The demo formats the outcome at its output boundary.

Example 03 Map changes a successful value

Use the supplied R.Map with an ordinary doubling function and observe that failures preserve their error.

```
Result<int> a = new Result<int>.Ok(3);
Result<int> b = new Result<int>.Fail("blocked");
var x = R.Map(a, n => n * 2);
var y = R.Map(b, n => n * 2);
Console.WriteLine(x is Result<int>.Ok { Value: 6 });
Console.WriteLine(y is Result<int>.Fail { Error: "blocked" });
```

Expected result

```
True
True
```

Explanation

1. The mapper returns int, not Result<int>. 2. Map applies it only to the Ok payload. 3. The failed source becomes a failed target with the same reason. 4. The operation does not print, open a file, or alter its source.

Example 04 Bind continues a fallible operation

Continue from a successful quantity to a stock check, which may itself reject the operation.

```
Result<int> input = new Result<int>.Ok(4);
var result = R.Bind(input, q => Reserve(q, 3));
Console.WriteLine(result is Result<int>.Fail { Error: "Insufficient stock" });

public static Result<int> Reserve(int q, int stock) => q <= stock
    ? new Result<int>.Ok(stock - q)
    : new Result<int>.Fail("Insufficient stock");
```

Expected result

```
True
```

Explanation

1. Bind receives a function returning Result<int>. 2. That function introduces a new possible domain failure. 3. Bind returns that result directly rather than nesting it. 4. The stock value remains input data; this calculation does not mutate inventory.

Example 05 Prove dependent work is skipped

Use a test-only counter to verify that a failed source does not call the dependent function.

```
int calls = 0;
Result<int> failed = new Result<int>.Fail("invalid");
var result = R.Bind(failed, x =>
{
    calls++;
    return new Result<int>.Ok(x + 1);
});
Console.WriteLine(calls);
Console.WriteLine(result is Result<int>.Fail { Error: "invalid" });
```

Expected result

```
0
True
```

Explanation

1. The source is already Fail. 2. Bind propagates the reason without invoking the delegate. 3. The counter belongs to the test observation, not to production domain calculation. 4. An assertion on calls detects an incorrectly eager implementation.

Practice tasks

Task 01 Zero is present

Basic | Fix code | Maybe

Fix Describe so Some(0) is not reported as missing. Handle None explicitly.

```
public static string Describe(Maybe<int> value) => value switch
{
    Maybe<int>.Some(var n) when n!=0 => $"Present:{n}",
    _ => "Missing"
};
```

Required checks Some(0) -> Present:0; Some(2) -> Present:2; None -> Missing.

Task 02 Find by identifier

Basic | Complete TODO | Maybe

Complete Find with exact ordinal matching; return Some for an existing Product and None otherwise.

```
public static Maybe<Product> Find(Product[] items,string id)
{
    // TODO lookup and return the appropriate case.
    throw new NotImplementedException();
}
```

Required checks P1 in catalogue -> Some(P1); P9 and empty catalogue -> None.

Task 03 Positive quantity Result

Basic | Implement | Result

Implement ValidateQuantity(int q) accepting 1 to 100 inclusive and returning Result<int>. Include useful failures.

Required checks 1 and 100 -> Ok; 0 and 101 -> Fail.

Task 04 Preserve a Map error

Basic | Fix code | Map

Fix Map so failure preserves its reason and does not return a successful zero.

```
public static Result<int> Map(Result<int> r,Func<int,int> f) => r switch
{
    Result<int>.Ok(var x) => new Result<int>.Ok(f(x)),
    _ => new Result<int>.Ok(0)
};
```

Required checks Ok 3 with doubling -> Ok 6; Fail("price") -> Fail("price").

Task 05 Complete generic Map

Basic | Complete TODO | Map

Complete Map<T, U>: map only Ok and propagate Fail into Result<U>. Reject unknown cases explicitly.

```
public static Result<U> Map<T,U>(Result<T> r,Func<T,U> f)
{
    // TODO match source cases and preserve failure.
    throw new NotImplementedException();
}
```
Required checks Ok 2 with x=>x.ToString() -> Ok"2"; failed source never invokes the mapper.

Task 06 Stock check as Result

Basic | Implement | Result

Implement CheckStock(Product p, int q), requiring q>0 and q<=Stock. Return the remaining stock as Ok without altering Product.

Required checks Stock 5/request 2 -> Ok 3; request 6 and 0 -> Fail; input stock remains 5.

Task 07 Do not unwrap a failure in Bind

Basic | Fix code | Bind

Fix Bind by handling Ok and Fail before reading a payload.

```
public static Result<int> Bind(Result<int> r,Func<int,Result<int>> f)
    => f(((Result<int>.Ok)r).Value);
```
Required checks Ok 2 continues; Fail("quantity") propagates without a cast exception or delegate call.

Task 08 Complete generic Bind

Basic | Complete TODO | Bind

Complete Bind<T, U> and return the next Result directly for Ok. Carry Fail through unchanged in meaning.

```
public static Result<U> Bind<T,U>(Result<T> r,Func<T,Result<U>> f)
{
    // TODO continue only for Ok.
    throw new NotImplementedException();
}
```
Required checks Ok 2 then reject-even -> Fail("even"); initial Fail("first") -> Fail("first") with zero calls.

Task 09 Maybe false is present

Basic | Implement | Maybe

Implement a formatter for Maybe<bool>. Some(false) means a recorded negative answer; None means no answer.

Required checks Some(false) -> Answer:no; Some(true) -> Answer:yes; None -> No answer.

Task 10 Avoid fabricated missing products

Basic | Fix code | Maybe

Fix Find so an absent id returns None instead of a zero-price Product.

```
public static Maybe<Product> Find(Product[] items,string id)
{
    var p=items.FirstOrDefault(x=>x.Id==id) ?? new Product(id,0,0);
    return new Maybe<Product>.Some(p);
}
```
Required checks P9 absent -> None; an actual P1 with price 0 remains Some.

Task 11 Require an optional item

Basic | Complete TODO | Maybe

Complete Require<T>(Maybe<T>, string error), converting Some to Ok and None to Fail.

```
public static Result<T> Require<T>(Maybe<T> value,string error)
{
    // TODO convert the two cases.
    throw new NotImplementedException();
}
```
Required checks Some 0 -> Ok 0; None with "missing" -> Fail("missing").

Task 12 Price validation

Basic | Implement | Result

Implement ValidatePrice(decimal p), accepting zero and positive prices; reject negative values without coercion.

Required checks 0 -> Ok 0; 2.5 -> Ok 2.5; -1 -> Fail.

Task 13 Use Bind for a fallible mapper

Basic | Fix code | Bind

Refactor the nested result to a single Result<int> using a continuation. Validate n>0.

```
public static Result<Result<int>> Validate(Result<int> source)
    => R.Map(source,n=>n>0
        ? (Result<int>)new Result<int>.Ok(n)
        : new Result<int>.Fail("positive"));
```

Required checks Ok 2 -> Ok 2; Ok 0 -> Fail("positive"); initial Fail("source") stays a single failure.

Task 14 Map to a view model

Basic | Complete TODO | Map

Complete ToQuote by mapping an accepted total to Quote(id, q, total). Validate id and q before this method is called.

```
public static Result<Quote> ToQuote(Result<decimal> total,string id,int q)
{
    // TODO use Map to create Quote.
    throw new NotImplementedException();
}
```

Required checks Ok 12, id P1, q 2 -> Ok Quote(P1,2,12); Fail("price") propagates.

Task 15 Counter proves short-circuiting

Basic | Implement | Bind

Write assertions for both Map and Bind using a test-only call counter. Test successful and failed sources separately.

Required checks Each Ok invokes its delegate once; each Fail invokes it zero times and preserves the error.

Task 16 Preserve the first failure

Basic | Fix code | Result

Fix CheckBoth so a failed quantity remains its original failure. Check price only after quantity succeeds.

```
public static Result<decimal> CheckBoth(Result<int> q,decimal p)
{
    if(p<0) return new Result<decimal>.Fail("price");
    return q is Result<int>.Ok(var n)
        ? new Result<decimal>.Ok(n*p) : new Result<decimal>.Fail("unknown");
}
```

Required checks Quantity failure "quantity" plus price failure "price" -> quantity; valid quantity plus bad price -> price.

Task 17 Map a present optional value

Basic | Complete TODO | Maybe

Complete MaybeMap<T, U> so Some invokes the mapper and None stays None.

```
public static Maybe<U> MaybeMap<T,U>(Maybe<T> source,Func<T,U> f)
{
    // TODO handle Some and None.
    throw new NotImplementedException();
}
```

Required checks Some 3 with +1 -> Some 4; None -> None with zero calls.

Task 18 Safe integer parsing

Basic | Implement | Result

Implement ParseQuantity(string text) using TryParse and a subsequent 1..100 check. Use different errors for format and range.

Required checks "2" -> Ok 2; "x" -> format failure; "0" -> range failure.

Task 19 Report failure instead of empty success

Basic | Fix code | Result

Fix Format so failure includes its original reason rather than reporting Total:0.

```
public static string Format(Result<decimal> r)
    => "Total:" + (r is Result<decimal>.Ok(var x) ? x : 0m);
```

Required checks Ok 0 -> Total:0; Fail("stock") -> Error:stock.

Task 20 Explicit optional name

Basic | Complete TODO | Maybe

Complete NameOrLabel: Some string is shown even when empty; None uses Missing. Do not apply domain name validation here.

```
public static string NameOrLabel(Maybe<string> name)
{
    // TODO handle presence independently of string content.
    throw new NotImplementedException();
}
```

Required checks Some("") -> Name:; Some("Ada") -> Name:Ada; None -> Missing.

Task 21 A two-check quote chain

Intermediate | Implement | Bind

Implement BuildQuote(p, q) by validating quantity, checking stock, and mapping the accepted request to a quote. Require non-negative price.

Required checks Stock 5, price 3, q 2 -> total 6; q 6 -> stock failure; negative price -> price failure; no input mutation.

Task 22 Do not call a mapper after Fail

Intermediate | Fix code | Map

Fix the eager implementation so a failed source never invokes f. Use a counter to prove it.

```
public static Result<int> Map(Result<int> r,Func<int,int> f)
{
    int value=r is Result<int>.Ok(var x) ? x : 0;
    int mapped=f(value);
    return r is Result<int>.Fail(var e)
        ? new Result<int>.Fail(e) : new Result<int>.Ok(mapped);
}
```

Required checks Fail("x") with mapper throwing if called -> Fail("x") without throw; Ok 2 -> correct mapping.

Task 23 Recover deliberately

Intermediate | Complete TODO | Result

Complete Recover(Result<int>, Func<string, int>): keep Ok as is; for Fail call the supplied recovery once and return Ok. Document this as an explicit caller policy.

```
public static Result<int> Recover(Result<int> r,Func<string,int> recovery)
{
    // TODO preserve Ok, recover Fail explicitly.
    throw new NotImplementedException();
}
```

Required checks Ok 4 -> 4/no recovery calls; Fail("missing") with recovery 42 -> Ok 42/one call.

Task 24 Collect a validated batch

Intermediate | Implement | Result

Implement Traverse(string[]) using ParseQuantity. Return all quantities only if every line succeeds; otherwise return the first error with its one-based line number.

Required checks [2,3] -> Ok[2,3]; [2, x,4] -> failure line 2 with no successful batch; [] -> Ok[].

Task 25 Keep a found zero-price item

Intermediate | Fix code | Maybe

Fix RequirePaidItem: a found free item is still present. Decide availability from the Maybe case, not its price.

```
public static Result<Product> RequirePaidItem(Maybe<Product> m)
    => m is Maybe<Product>.Some(var p) && p.Price>0
        ? new Result<Product>.Ok(p) : new Result<Product>.Fail("missing");
```

Required checks Some(P1, stock 2, price 0) -> Ok Product; None -> Fail("missing").

Task 26 Bind a missing optional lookup

Intermediate | Complete TODO | Maybe

Complete MaybeBind<T, U> so a present item can lead to another optional lookup; propagate None without calling the function.

```
public static Maybe<U> MaybeBind<T,U>(Maybe<T> m,Func<T,Maybe<U>> f)
{
    // TODO continue only when Some.
    throw new NotImplementedException();
}
```

Required checks Some P1 then absent supplier -> None; initial None -> zero calls.

Task 27 Lookup then validate then price

Intermediate | Implement | Bind

Implement a chain from required product lookup to validated quantity and total. Use Bind for checks and Map for final projection. Return distinct missing, quantity, stock errors.

Required checks P1 stock 3/price 4, q 2 -> total 8; absent P9 -> missing; q 0 -> quantity; q 4 -> stock.

Task 28 Do not hide a missing file

Intermediate | Fix code | Result

Fix the file boundary to return Fail with the path context when reading fails; keep successful empty content as Ok("").

```
public static Result<string> Read(string path)
{
    try { return new Result<string>.Ok(File.ReadAllText(path)); }
    catch(IOException) { return new Result<string>.Ok(""); }
}
```

Required checks Existing empty file -> Ok empty string; missing file -> Fail containing path.

Task 29 Convert operational failure at one boundary

Intermediate | Complete TODO | Result

Complete ReadLines with relevant IOException and UnauthorizedAccessException handling. Return Result<string[]>; avoid a catch-all claiming success.

```
public static Result<string[]> ReadLines(string path)
{
    // TODO read once and handle relevant operational exceptions.
    throw new NotImplementedException();
}
```

Required checks File A, B -> Ok[A, B]; missing -> clear Fail; pure downstream parser receives lines only on Ok.

Task 30 Compare exception and Result contracts

Intermediate | Implement | Result

Implement the same pure positive-quantity rule as a throwing method and a Result method. Write assertions for valid and invalid inputs and explain how callers see failure in each API.

Required checks 2 succeeds in both; 0 is a domain rejection; Result version does not throw for 0.

Task 31 Propagate the next operation error

Intermediate | Fix code | Bind

Fix Bind so a failure from f is not replaced with an unrelated error.

```
public static Result<int> Bind(Result<int> r,Func<int,Result<int>> f)
{
    if(r is Result<int>.Fail(var e)) return new Result<int>.Fail(e);
    var next=f(((Result<int>.Ok)r).Value);
    return next is Result<int>.Ok ? next : new Result<int>.Fail("failed");
}
```

Required checks Ok 2 then Fail("limit") -> limit; initial Fail("input") -> input.

Task 32 Compose two independent successful values

Intermediate | Complete TODO | Map

Complete Combine(a, b, f) producing Result<U>. If a fails return its error; otherwise if b fails return its error; invoke f only when both succeed.

```
public static Result<U> Combine<T,U>(Result<T> a,Result<T> b,Func<T,T,U> f)
{
    // TODO apply only after both successful cases.
    throw new NotImplementedException();
}
```

Required checks Ok 2, Ok 3,+ -> Ok 5; Fail A, Fail B -> A; one failed -> zero calls.

Task 33 Map a successful collection

Intermediate | Implement | Map

Implement Map over Result<Product[]> to return an ordered Quote[] with total Stock*Price and ties by ordinal id. Preserve input and source errors.

Required checks P2:2*3, P1:3*2 -> P1, P2 totals 6; failed catalogue -> same failure; empty -> Ok[].

Task 34 Maybe absence without null suppression

Intermediate | Fix code | Maybe

Fix GetStock: return Result<int> and handle None with a missing error; present Stock 0 must return Ok 0.

```
public static int GetStock(Maybe<Product> item)
    => ((item as Maybe<Product>.Some)!).Value.Stock;
```

Required checks Some(P1,0,3) -> Ok 0; None -> Fail("missing").

Task 35 Add context without losing the error

Intermediate | Complete TODO | Result

Complete WithContext<T>(r, context): preserve Ok; prefix failure with context and original reason.

```
public static Result<T> WithContext<T>(Result<T> r,string context)
{
    // TODO transform only the error case.
    throw new NotImplementedException();
}
```

Required checks Fail("quantity"),"line 2" -> Fail("line 2:quantity"); Ok 3 unchanged.

Task 36 Error accumulation is a different policy

Intermediate | Implement | Result

Implement ValidateProduct(id, stock, price) collecting all independent field errors in fixed id/stock/price order. Return Product only when no errors exist; explain how this differs from fail-fast Bind.

Required checks blank,-1,-2 -> three errors in order; P1,0,0 -> Ok Product.

Task 37 Correctly distinguish an empty result

Intermediate | Fix code | Maybe

Fix FirstMatch so a matching Product with Stock 0 is Some. Missing means no matching id.

```
public static Maybe<Product> FirstMatch(Product[] ps,string id)
{
    var p=ps.FirstOrDefault(x=>x.Id==id);
    return p!=null && p.Stock>0
        ? new Maybe<Product>.Some(p) : new Maybe<Product>.None();
}
```

Required checks P1 with stock 0 -> Some; P9 absent -> None.

Task 38 Sequence a lazy source safely

Intermediate | Complete TODO | Bind

Complete Sequence(IEnumerable<Result<int>>) returning an array on success or the first failure. Stop enumeration immediately at the first failure.

```
public static Result<int[]> Sequence(IEnumerable<Result<int>> source)
{
    // TODO enumerate once and stop at first Fail.
    throw new NotImplementedException();
}
```

Required checks Ok 1, Fail x, third producer -> Fail x; third producer not visited; [] -> Ok[].

Task 39 Prove a required lookup stops later checks

Intermediate | Implement | Bind

Build a required lookup chain with test-only counters for validation and pricing. Check both a missing item and an item failing validation.

Required checks Missing -> validation 0/pricing 0; invalid quantity -> validation 1/pricing 0; valid -> each 1.

Task 40 Lift a primitive transformation without a default

Intermediate | Fix code | Map

Fix Convert so Fail does not become an Ok string derived from zero; preserve the reason while changing payload type.

```
public static Result<string> Convert(Result<int> r)
    => new Result<string>.Ok((r is Result<int>.Ok(var x)?x:0).ToString());
```

Required checks Ok 0 -> Ok"0"; Fail("absent") -> Fail("absent").

Task 41 Dependent validation with an accumulated first phase

Advanced | Complete TODO | Bind

Complete Build using independent input-field validation first, then Bind a stock check and Map a quote. Preserve deterministic error order and do not invoke stock checks after invalid fields.

```
public static Result<Quote> Build(string id,int quantity,decimal price,Product[] items)
{
    // TODO accumulate input errors, then continue only after success.
    throw new NotImplementedException();
}
```

Required checks Two invalid fields -> ordered errors/no stock call; valid fields with insufficient stock -> stock failure.

Task 42 A generic Maybe to Result boundary

Advanced | Implement | Maybe

Implement Require<T> with a supplied Func<string> error factory. Compute the error lazily only for None. Include Some(false), Some 0 and Some empty string checks.

Required checks Some 0 -> Ok 0/errorFactory 0 calls; None -> Fail generated once.

Task 43 Validate before reserving

Advanced | Fix code | Bind

Fix the routine so state changes occur only after validation and pricing both succeed. Return a new Product and Quote rather than changing the input list early.

```
public static Result<decimal> Reserve(List<Product> items,int q)
{
    var p=items[0]; items[0]=p with { Stock=p.Stock-q };
    if(q<=0) return new Result<decimal>.Fail("quantity");
    if(p.Price<0) return new Result<decimal>.Fail("price");
    return new Result<decimal>.Ok(q*p.Price);
}
```

Required checks Stock 5, q 2, price 3 -> next stock 3/total 6; pricing failure leaves original stock 5.

Task 44 Trace an outcome chain

Advanced | Complete TODO | Bind

Complete Trace: return pure stage labels alongside the outcome for parse, validate and price. Failed stages stop later stage labels; test counters may supplement the trace.

```
public static (Result<decimal> Outcome,string[] Stages) Trace(string text,decimal price)
{
    // TODO return a deterministic trace together with the final outcome.
    throw new NotImplementedException();
}
```

Required checks "2", price 3 -> parse, validate, price/6; "x" -> parse only/format failure; "0" -> parse, validate/range failure.

Task 45 Compare two error-handling implementations

Advanced | Implement | Map

Implement equivalent nested case handling and a Map/Bind chain for lookup-validation-pricing. Assert agreement on success and every failure; compare clarity and dependency visibility without timing claims.

Required checks Use missing id, zero quantity, insufficient stock, and a normal order with documented outputs.

Task 46 Inventory purchase chain

Advanced | Integrate | Bind

Build a required lookup, quantity validation, stock check, and quote projection. Return a proposed new inventory snapshot only after success. Add first-failure and no-dependent-call assertions.

Required checks P1 stock 5/price 3, q 2 -> stock 3, total 6; absent item or q 6 -> failure with unchanged input.

Task 47 File import with explicit errors

Advanced | Integrate | Result

Combine a file boundary, line parser, and batch validator using Result. Missing file and malformed row must be distinct errors; malformed row identifies its line. Publish no partial batch.

Required checks Valid two-line file -> 2 products; malformed second line -> failure line 2; missing -> file failure.

Task 48 Optional profile fields

Advanced | Integrate | Maybe

Model optional age and consent separately from validation. Some 0 is present but may be invalid under the age rule; Some(false) is a present refusal. Build a registration Result chain with explicit missing and rejected outcomes.

Required checks Age missing -> missing-age; age 0 -> age-rule; consent false -> refusal; valid age+true -> success.

Task 49 Quote report from a validated dataset

Advanced | Integrate | Map

Import products into Result<Product[]> then use Map to build a deterministic total-by-price report. Use Bind for a later export boundary. Failed import must prevent report and export work.

Required checks Products P1 stock 2/price 3, P2 stock 1/price 4 -> total 10; failed import -> zero export calls.

Task 50 Refactor a project error path

Advanced | Integrate | Bind

Choose one exception-heavy or default-returning domain workflow. Introduce at least three Result failures, one Maybe lookup, and used Map/Bind. Document before/after behaviour and prove consistent state after rejection.

Required checks Provide concrete normal, absent, invalid, and operational-failure demonstrations, plus assertions on skipped dependent work.